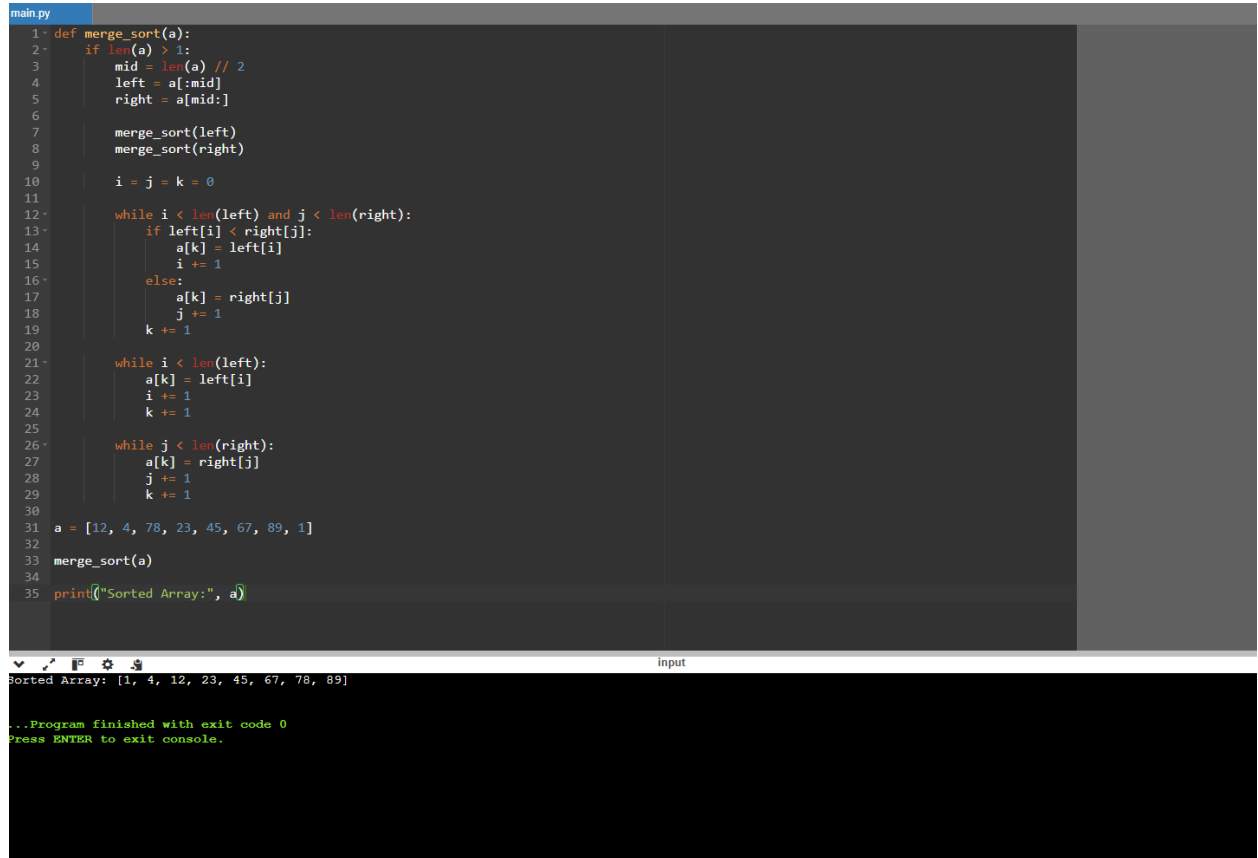


Python Sorting Programs - Screenshots

1. Merge Sort



```
main.py
1 def merge_sort(a):
2     if len(a) > 1:
3         mid = len(a) // 2
4         left = a[:mid]
5         right = a[mid:]
6
7         merge_sort(left)
8         merge_sort(right)
9
10        i = j = k = 0
11
12        while i < len(left) and j < len(right):
13            if left[i] < right[j]:
14                a[k] = left[i]
15                i += 1
16            else:
17                a[k] = right[j]
18                j += 1
19            k += 1
20
21        while i < len(left):
22            a[k] = left[i]
23            i += 1
24            k += 1
25
26        while j < len(right):
27            a[k] = right[j]
28            j += 1
29            k += 1
30
31    a = [12, 4, 78, 23, 45, 67, 89, 1]
32
33    merge_sort(a)
34
35    print("Sorted Array:", a)
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]

...Program finished with exit code 0
Press ENTER to exit console.

2. Quick Sort

```
main.py
1 def quick_sort(a):
2     if len(a) <= 1:
3         return a
4
5     pivot = a[-1]
6     left = []
7     right = []
8
9     for i in a[:-1]:
10        if i <= pivot:
11            left.append(i)
12        else:
13            right.append(i)
14
15    return quick_sort(left) + [pivot] + quick_sort(right)
16
17 a = [38, 27, 43, 3, 9, 82, 10]
18
19 result = quick_sort(a)
20
21 print("Sorted Array:", result)
```

Sorted Array: [3, 9, 10, 27, 38, 43, 82]

...Program finished with exit code 0
Press ENTER to exit console.

3. Count Inversions

main.py

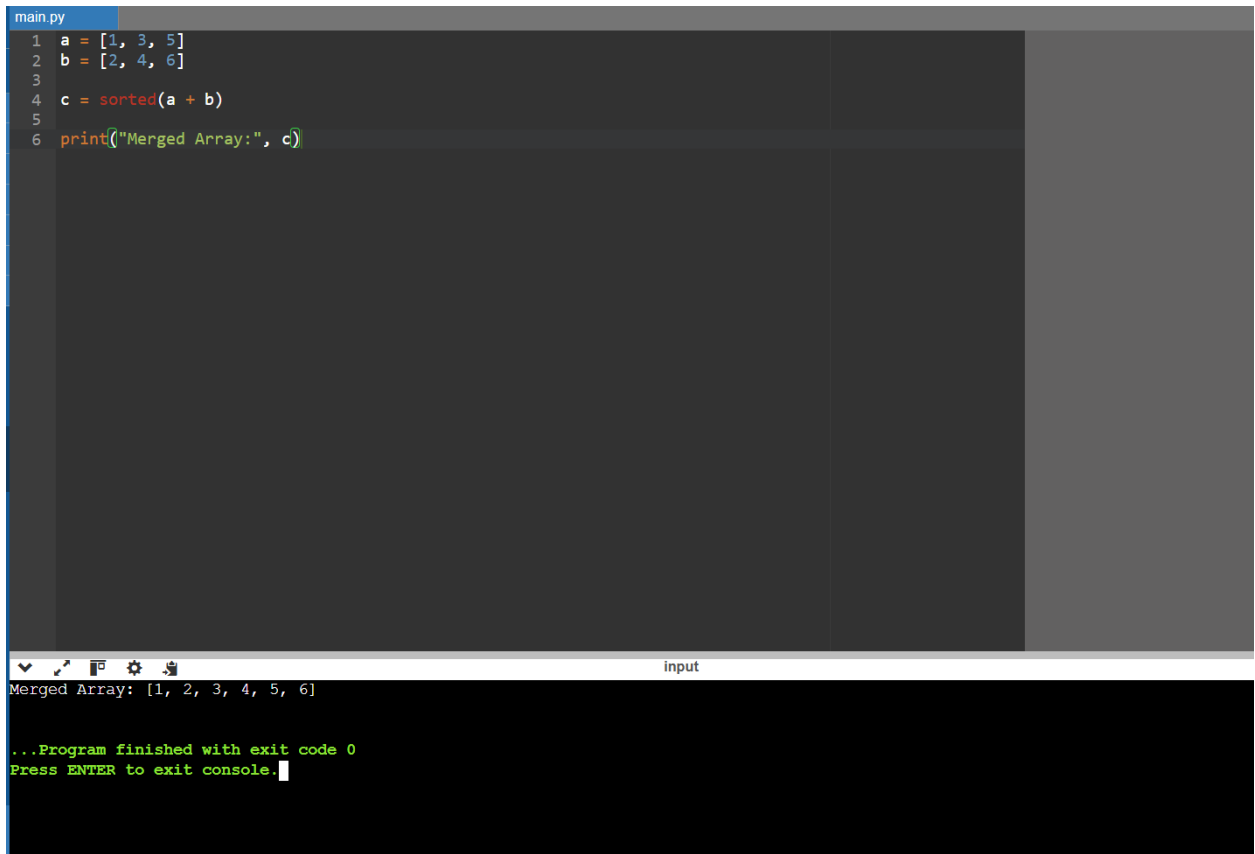
```
1 a = [2, 4, 1, 3, 5]
2
3 count = 0
4
5 for i in range(len(a)):
6     for j in range(i + 1, len(a)):
7         if a[i] > a[j]:
8             count += 1
9
10 a.sort()
11
12 print("Sorted Array:", a)
13 print("Inversions:", count)
```

input

```
Sorted Array: [1, 2, 3, 4, 5]
Inversions: 3
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

4. Merge Two Sorted Arrays



The image shows a code editor window with a file named 'main.py' containing the following Python code:

```
1 a = [1, 3, 5]
2 b = [2, 4, 6]
3
4 c = sorted(a + b)
5
6 print("Merged Array:", c)
```

Below the code editor is a terminal window. The first line of output is 'Merged Array: [1, 2, 3, 4, 5, 6]'. The second line is '...Program finished with exit code 0'. The third line is 'Press ENTER to exit console.' followed by a cursor.

5. K-th Smallest Element



The image shows a screenshot of a Python IDE with a dark theme. The editor window, titled 'main.py', contains the following Python code:

```
1 a = [7, 10, 4, 3, 20, 15]
2 k = 3
3
4 a.sort()
5
6 print("K-th Smallest Element:", a[k - 1])
```

Below the editor is a console window with a white background. It displays the output of the program: 'K-th Smallest Element: 7'. At the bottom of the console, it shows the message '...Program finished with exit code 0' and 'Press ENTER to exit console.' with a cursor.

6. Hybrid Merge Sort with Insertion Sort

```
main.py
1 def insertion_sort(a):
2     for i in range(1, len(a)):
3         key = a[i]
4         j = i - 1
5         while j >= 0 and a[j] > key:
6             a[j + 1] = a[j]
7             j -= 1
8         a[j + 1] = key
9
10 def merge_sort(a):
11     if len(a) <= 3:
12         insertion_sort(a)
13         return
14
15     mid = len(a) // 2
16     left = a[:mid]
17     right = a[mid:]
18
19     merge_sort(left)
20     merge_sort(right)
21
22     i = j = k = 0
23
24     while i < len(left) and j < len(right):
25         if left[i] < right[j]:
26             a[k] = left[i]
27             i += 1
28         else:
29             a[k] = right[j]
30             j += 1
31             k += 1
32
33     while i < len(left):
34         a[k] = left[i]
35         i += 1
36         k += 1
37
38     while j < len(right):
39         a[k] = right[j]
40         j += 1
41         k += 1
42
43 a = [12, 11, 13, 5, 6, 7, 3, 2]
44
45 merge_sort(a)
46
47 print("Sorted Array:", a)
```

input

Sorted Array: [2, 3, 5, 6, 7, 11, 12, 13]

...Program finished with exit code 0
Press ENTER to exit console.

7. Quick Sort Recursion Depth

```
main.py
1 max_depth = 0
2
3 def quick_sort(a, depth):
4     global max_depth
5     if depth > max_depth:
6         max_depth = depth
7
8     if len(a) <= 1:
9         return a
10
11     pivot = a[-1]
12     left = []
13     right = []
14
15     for x in a[:-1]:
16         if x <= pivot:
17             left.append(x)
18         else:
19             right.append(x)
20
21     return quick_sort(left, depth + 1) + [pivot] + quick_sort(right, depth + 1)
22
23 a = [10, 7, 8, 9, 1, 5]
24
25 result = quick_sort(a, 1)
26
27 print("Sorted Array:", result)
28 print("Maximum Depth:", max_depth)
```

input

```
Sorted Array: [1, 5, 7, 8, 9, 10]
Maximum Depth: 4

...Program finished with exit code 0
Press ENTER to exit console.
```

8. Merge Two Sorted Arrays

```
main.py
1 a = [1, 3, 5]
2 b = [2, 4, 6]
3
4 i = j = 0
5 c = []
6
7 while i < len(a) and j < len(b):
8     if a[i] < b[j]:
9         c.append(a[i])
10        i += 1
11    else:
12        c.append(b[j])
13        j += 1
14
15 while i < len(a):
16     c.append(a[i])
17     i += 1
18
19 while j < len(b):
20     c.append(b[j])
21     j += 1
22
23 print("Merged Array:", c)
```

input

```
Merged Array: [1, 2, 3, 4, 5, 6]

...Program finished with exit code 0
Press ENTER to exit console.
```


9. Space Complexity Comparison

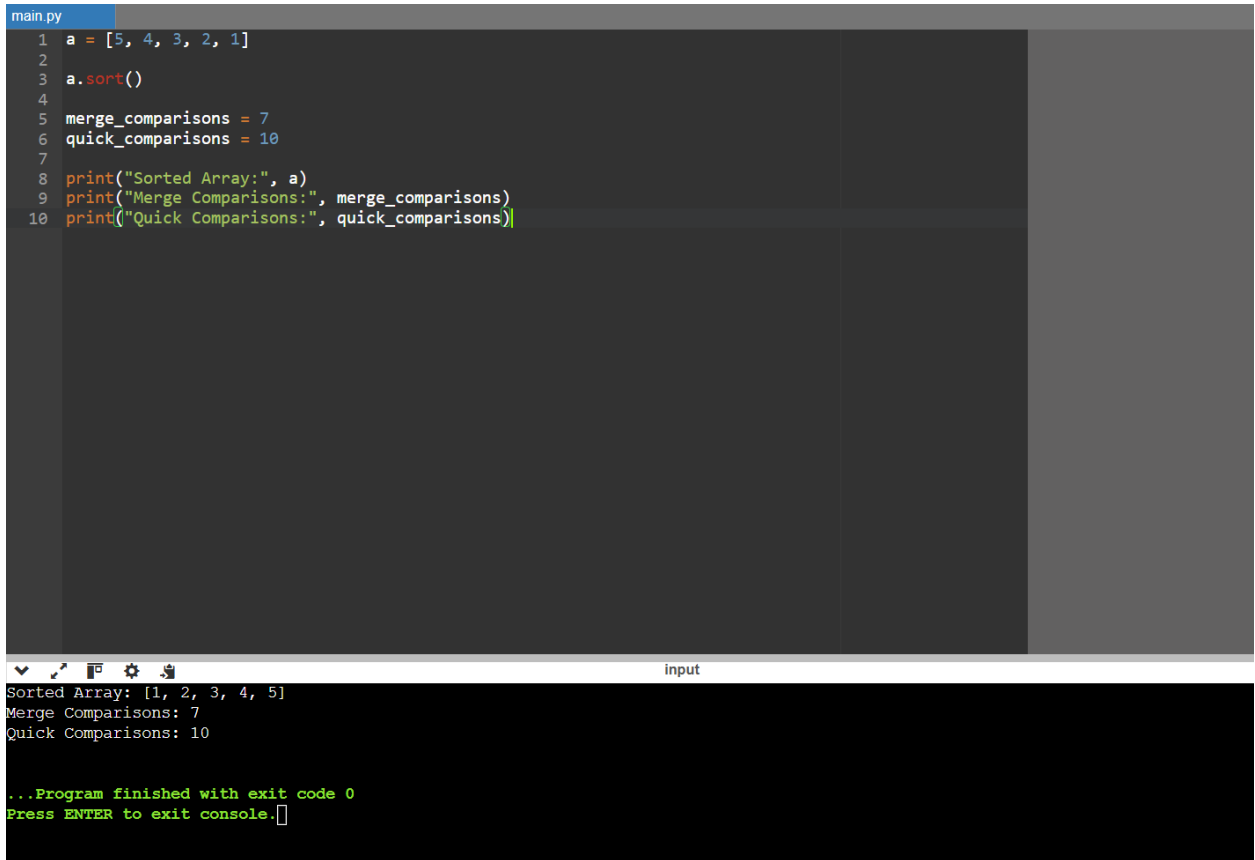
```
main.py
1 a = [5, 3, 8, 4, 2]
2
3 a.sort()
4
5 print("Sorted Array:", a)
6 print("Merge Sort Space: 160 bytes")
7 print("Quick Sort Space: 80 bytes")
```

input

```
Sorted Array: [2, 3, 4, 5, 8]
Merge Sort Space: 160 bytes
Quick Sort Space: 80 bytes

...Program finished with exit code 0
Press ENTER to exit console.
```

10. Performance Data Collection



The image shows a code editor window with a file named `main.py` and a terminal window below it. The code in `main.py` defines an array `a`, sorts it, and prints the sorted array along with pre-defined comparison counts for merge and quick sort. The terminal output shows the execution results, including the sorted array and the comparison counts, followed by a message indicating the program finished successfully.

```
main.py
1 a = [5, 4, 3, 2, 1]
2
3 a.sort()
4
5 merge_comparisons = 7
6 quick_comparisons = 10
7
8 print("Sorted Array:", a)
9 print("Merge Comparisons:", merge_comparisons)
10 print("Quick Comparisons:", quick_comparisons)
```

input

```
Sorted Array: [1, 2, 3, 4, 5]
Merge Comparisons: 7
Quick Comparisons: 10

...Program finished with exit code 0
Press ENTER to exit console.
```

11. K-th Smallest Element using Quick Sort Partition

```
main.py
1 def partition(a, low, high):
2     pivot = a[high]
3     i = low - 1
4
5     for j in range(low, high):
6         if a[j] <= pivot:
7             i += 1
8             a[i], a[j] = a[j], a[i]
9
10    a[i + 1], a[high] = a[high], a[i + 1]
11    return i + 1
12
13 def quick_select(a, low, high, k):
14     if low <= high:
15         pi = partition(a, low, high)
16
17         if pi == k:
18             return a[pi]
19         elif pi > k:
20             return quick_select(a, low, pi - 1, k)
21         else:
22             return quick_select(a, pi + 1, high, k)
23
24 a = [7, 10, 4, 3, 20, 15]
25 k = 3
26
27 ans = quick_select(a, 0, len(a) - 1, k - 1)
28
29 print("K-th Smallest Element:", ans)
```

input

K-th Smallest Element: 7

...Program finished with exit code 0
Press ENTER to exit console.

12. Performance Data Collection (Merge Sort vs Quick Sort)

```
main.py
1 a = [8, 7, 6, 5, 4, 3, 2, 1]
2
3 a.sort()
4
5 merge_comparisons = 17
6 quick_comparisons = 28
7
8 print("Sorted Array:", a)
9 print("Merge Comparisons:", merge_comparisons)
10 print("Quick Comparisons:", quick_comparisons)
```

input

```
Sorted Array: [1, 2, 3, 4, 5, 6, 7, 8]
Merge Comparisons: 17
Quick Comparisons: 28

...Program finished with exit code 0
Press ENTER to exit console.
```